



Product Verification System

An full-stack web application for warehouse product lifecycle management. Supports bulk CSV ingestion, AI-powered on-floor product verification with OCR, role-based access control, and comprehensive reporting.



Table of Contents

1. [Overview](#)
2. [Key Features](#)
3. [Application Preview](#)
4. [Architecture & Design Decisions](#)
5. [Technology Stack](#)
6. [Project Structure](#)
7. [Data Schema](#)
8. [API Reference](#)
9. [Role-Based Access Control \(RBAC\)](#)
10. [OCR Pipeline](#)
11. [Prerequisites](#)
12. [Environment Configuration](#)
13. [Installation & Setup](#)
14. [Running the Application](#)
15. [CSV Format Specification](#)
16. [User Guide](#)
17. [Scalability & Performance](#)
18. [Security Considerations](#)
19. [Troubleshooting](#)
20. [License](#)

Overview

The **Product Verification System** is built to solve a core challenge in warehouse and logistics operations: maintaining accurate, verifiable records of physical product lifecycle data (manufacturing dates, expiry dates) across thousands or millions of SKUs.

The system is designed for **three distinct user personas**:

Persona	Role	Primary Workflow
Warehouse Manager	admin	Upload bulk CSV inventory data; manage users
Warehouse Operator	operator	Scan or photograph products on the floor to verify them
Quality Assurance Manager	qa	Generate date-range reports on all verification activity

All three personas interact through a single, unified web application with role-aware navigation — each user sees only the tools relevant to their job.

Key Features

Core Features (Mandatory)

- **Bulk CSV Ingestion** — Upload millions of product records via CSV. Processing runs as a background task with live progress polling and a cancel option. Duplicate WIDs are upserted, not duplicated.
- **On-Floor Product Validation** — Operators input a WID (manually or via barcode scanner) and optionally capture a product photo. The system looks up the product in the database and, if an image is provided, runs OCR to extract dates from the physical label and compares them.
- **Verification Reporting** — QA managers select a date range and retrieve a full log of all validation events, including operator, timestamp, status, OCR results, and captured image paths. Reports are exportable as **CSV** or **PDF**.

Optional / Bonus Features (All Implemented)

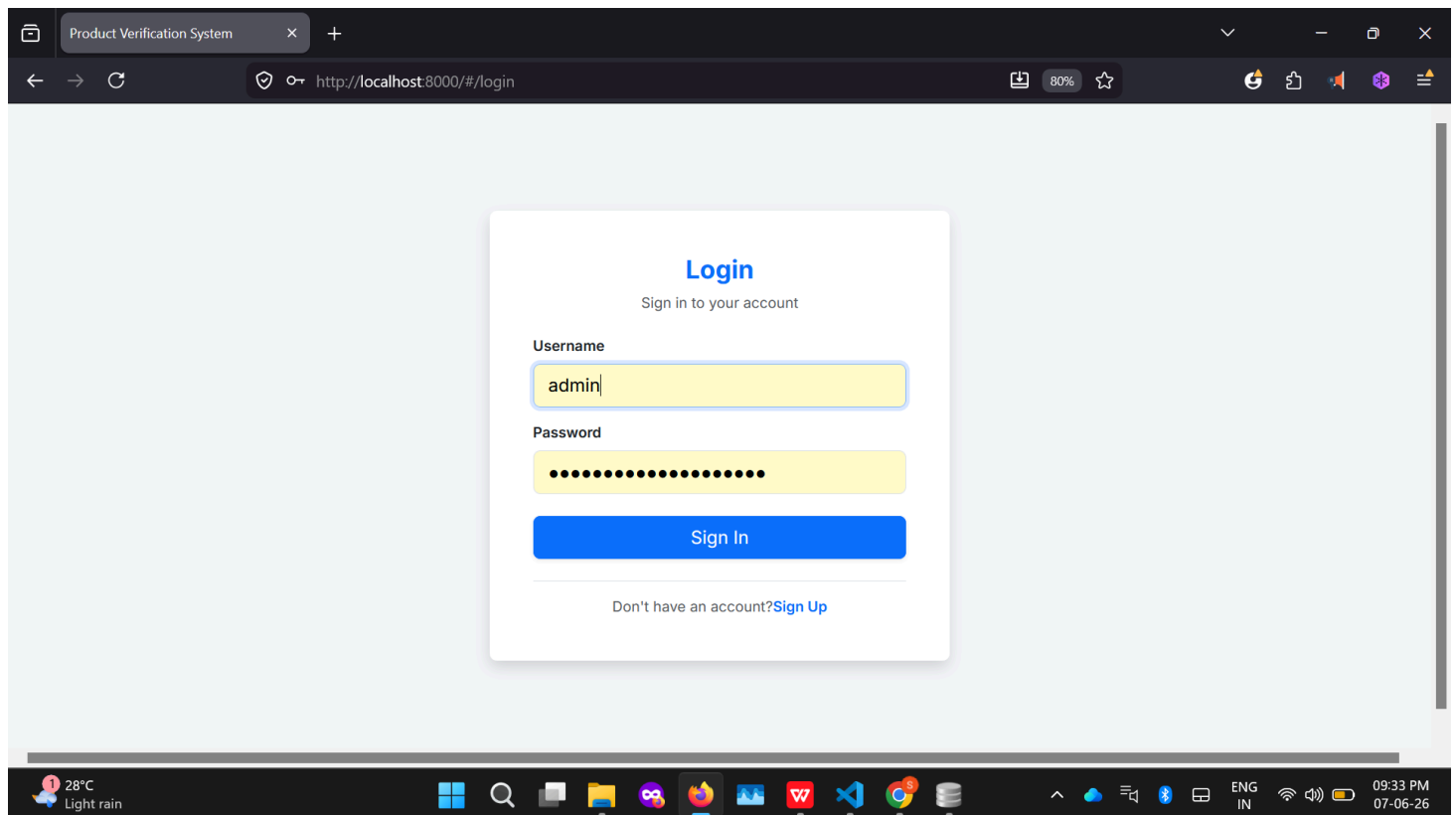
- **User Login & Signup** — JWT-based authentication with access tokens and persistent refresh tokens.
- **Admin User Management** — Admins can create, view, and delete users; assign any of the three roles (admin , operator , qa).
- **Role-Based Access Control** — Frontend routing and backend API endpoints are both role-guarded. Unauthorized navigation attempts redirect gracefully.
- **AI-Powered Date Extraction from Images** — A two-stage OCR pipeline extracts manufacturing and expiry dates from product photos automatically, validating them against the database record.

Application Preview

This section provides a visual walkthrough of the Product Verification System and its key workflows.

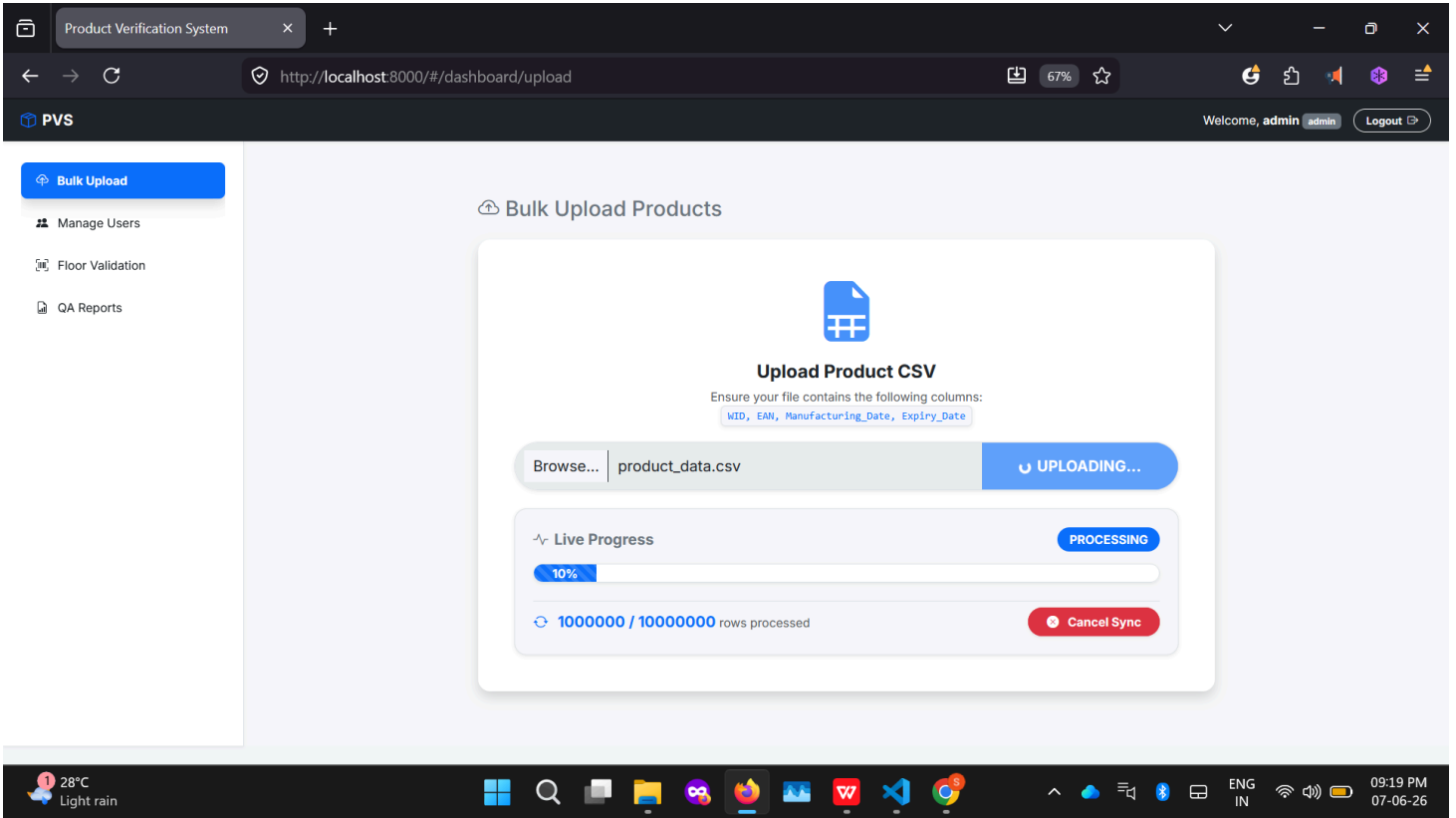
Login Screen

The authentication page allows users to securely sign in using their assigned credentials. JWT-based authentication is used for access control and session management.



Bulk Product Upload

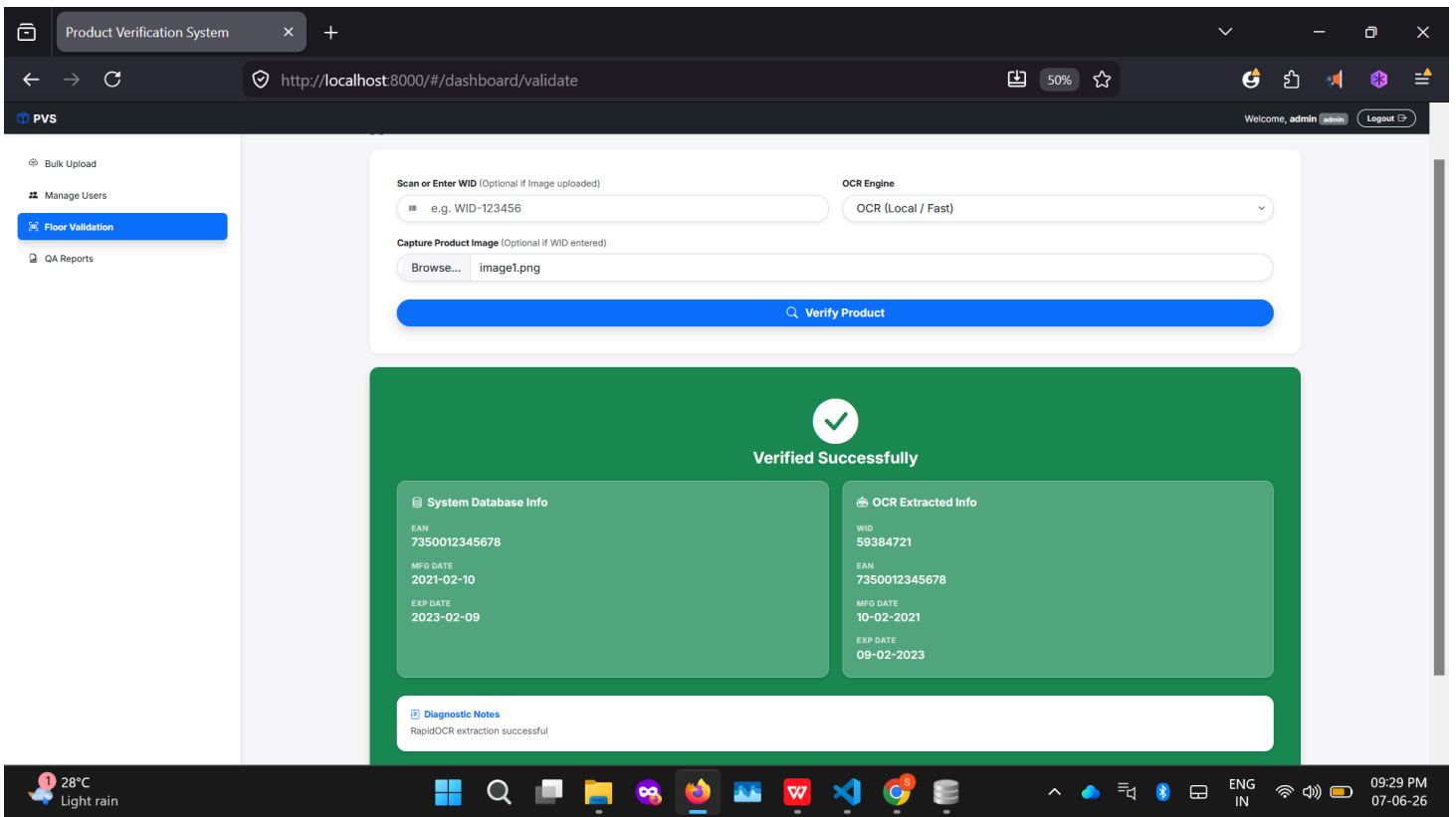
Administrators can upload large CSV files containing product master data. Uploads are processed asynchronously with progress tracking and cancellation support.



The screenshot displays the 'Bulk Upload Products' interface within the 'Product Verification System' (PVS) web application. The browser's address bar shows the URL `http://localhost:8000/#/dashboard/upload`. The application header includes the 'PVS' logo and a user greeting 'Welcome, admin' with a 'Logout' button. A left sidebar contains navigation links: 'Bulk Upload' (active), 'Manage Users', 'Floor Validation', and 'QA Reports'. The main content area features a central card titled 'Bulk Upload Products' with a CSV icon and the heading 'Upload Product CSV'. Below this, a message states: 'Ensure your file contains the following columns: WID, EAN, Manufacturing_Date, Expiry_Date'. A file upload section shows a 'Browse...' button, the filename 'product_data.csv', and a blue 'UPLOADING...' button. A 'Live Progress' section displays a 'PROCESSING' status, a progress bar at 10%, and a counter '1000000 / 10000000 rows processed'. A red 'Cancel Sync' button is located at the bottom right of the progress section. The system tray at the bottom shows a temperature of 28°C, 'Light rain' weather, and the time 09:19 PM on 07-06-26.

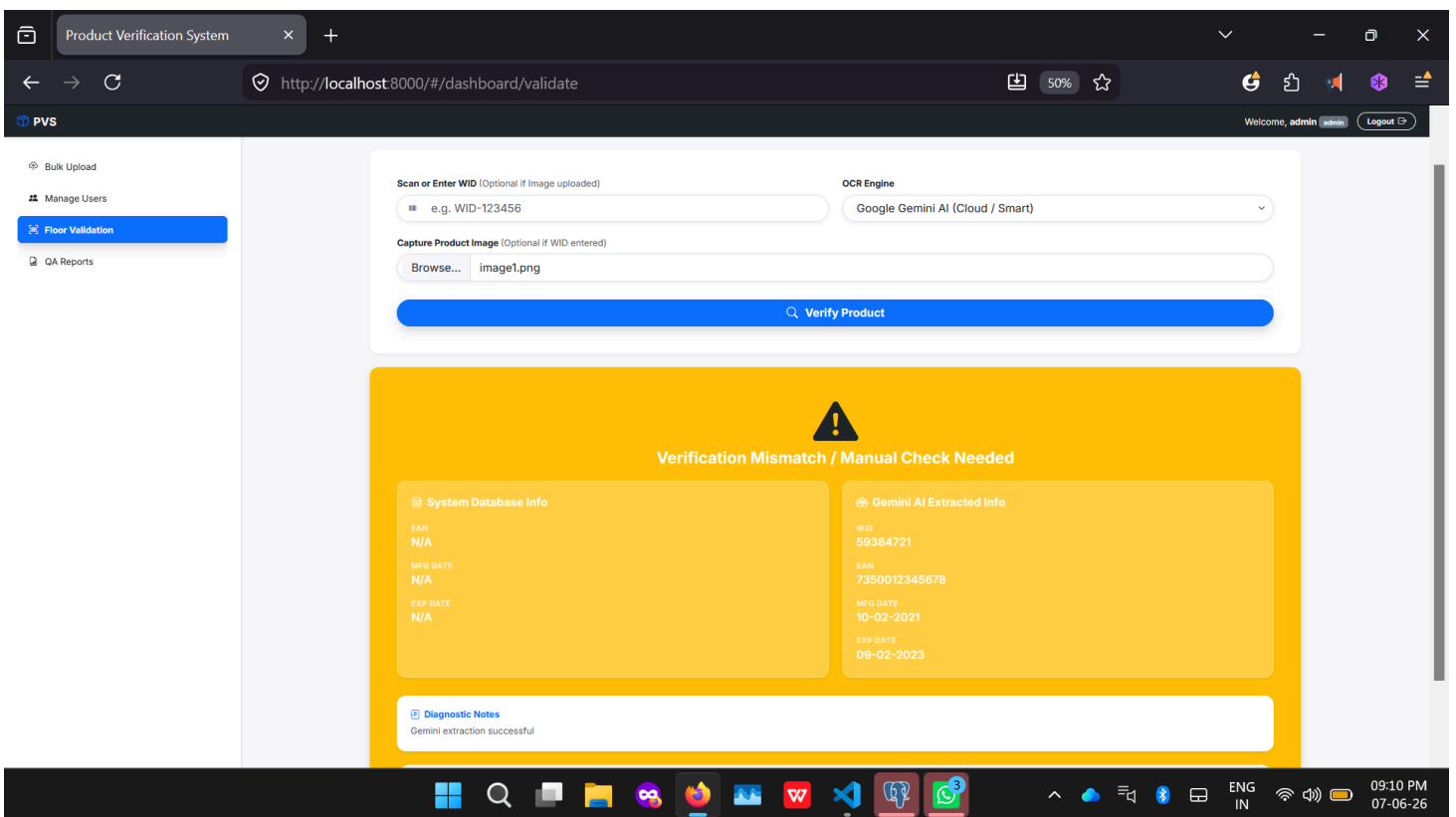
Product Verification using Local OCR

Operators can validate products by entering/scanning a WID and uploading a product image. The system extracts manufacturing and expiry dates using the local OCR engine and compares them with warehouse records.



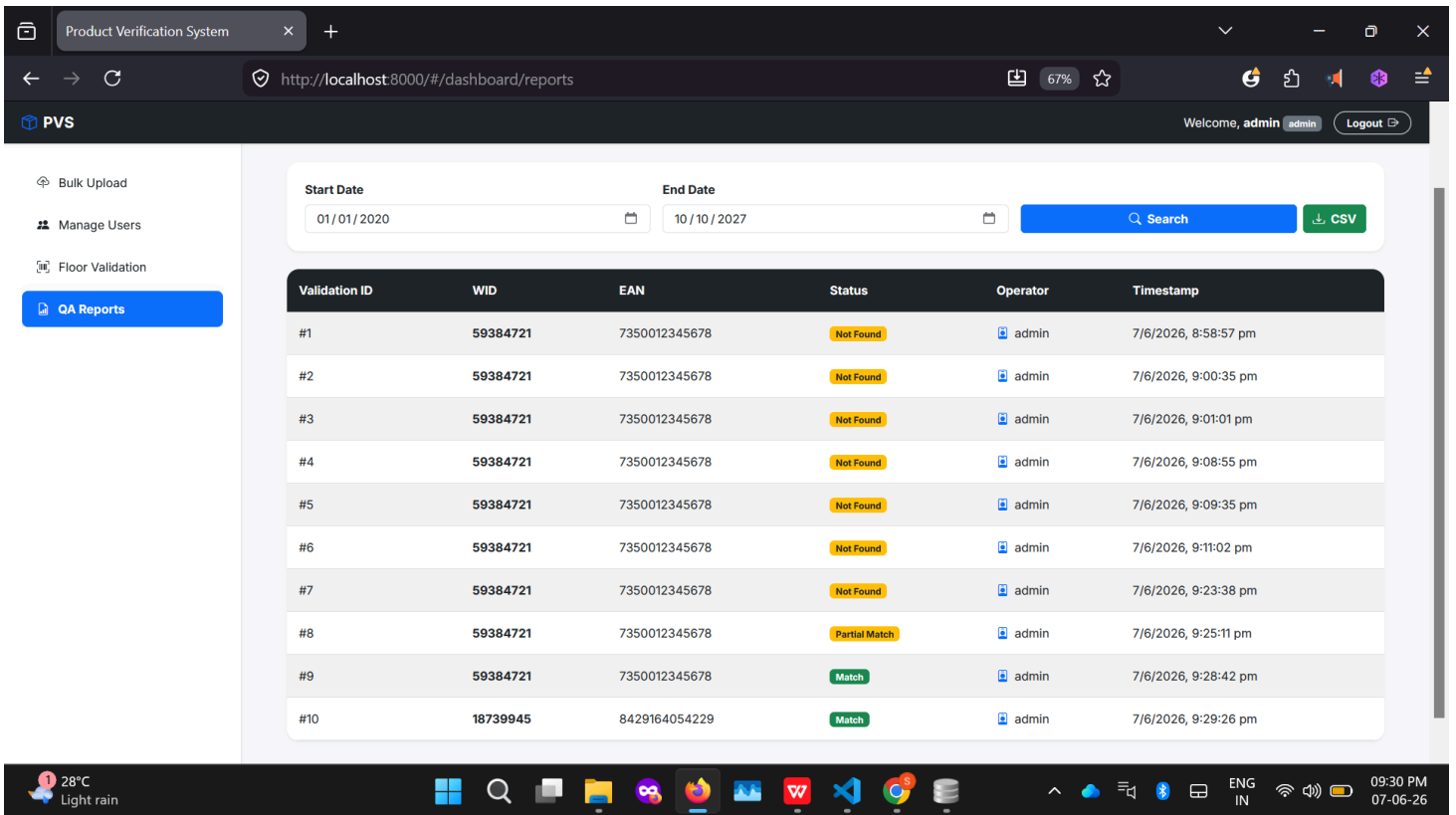
Product Verification using Gemini AI OCR

For complex labels where traditional OCR may struggle, the system can leverage Google Gemini Vision to intelligently extract structured product information.



Verification Reports & Analytics

QA Managers can generate date-range based reports containing validation history, verification outcomes, OCR results, timestamps, and operator activity. Reports can be exported in CSV or PDF format.

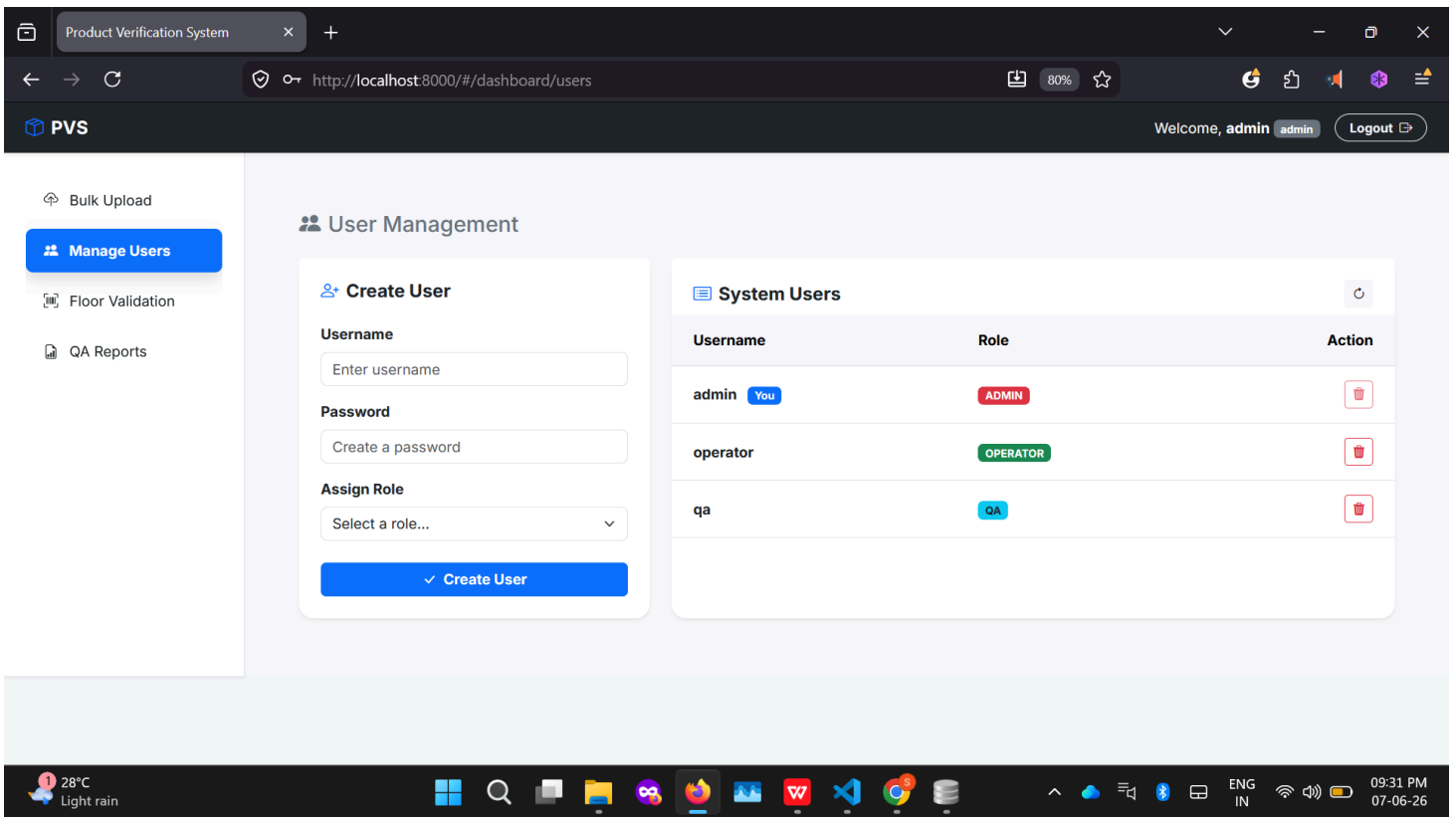


The screenshot displays the Product Verification System (PVS) dashboard. The top navigation bar includes the PVS logo, a user profile for 'admin', and a 'Logout' button. The left sidebar contains links for 'Bulk Upload', 'Manage Users', 'Floor Validation', and 'QA Reports' (highlighted in blue). The main content area features a date range filter (Start Date: 01/01/2020, End Date: 10/10/2027) and a 'Search' button. Below the filter is a table with 10 rows of validation data.

Validation ID	WID	EAN	Status	Operator	Timestamp
#1	59384721	7350012345678	Not Found	admin	7/6/2026, 8:58:57 pm
#2	59384721	7350012345678	Not Found	admin	7/6/2026, 9:00:35 pm
#3	59384721	7350012345678	Not Found	admin	7/6/2026, 9:01:01 pm
#4	59384721	7350012345678	Not Found	admin	7/6/2026, 9:08:55 pm
#5	59384721	7350012345678	Not Found	admin	7/6/2026, 9:09:35 pm
#6	59384721	7350012345678	Not Found	admin	7/6/2026, 9:11:02 pm
#7	59384721	7350012345678	Not Found	admin	7/6/2026, 9:23:38 pm
#8	59384721	7350012345678	Partial Match	admin	7/6/2026, 9:25:11 pm
#9	59384721	7350012345678	Match	admin	7/6/2026, 9:28:42 pm
#10	18739945	8429164054229	Match	admin	7/6/2026, 9:29:26 pm

User Management

Administrators can create, view, and manage users across different roles (admin , operator , and qa) through the centralized user management interface.



Architecture & Design Decisions

Single-Binary Deployment

The entire application (backend API + frontend static files) is served by a **single Uvicorn/FastAPI process**. The Vue.js frontend is a set of static `.js` files that FastAPI mounts and serves directly. This eliminates the need for a separate Node.js server or a reverse proxy in development, simplifying deployment and reducing infrastructure overhead.

Background Task Processing for CSV Uploads

Large CSVs (millions of rows) cannot be processed in a single synchronous HTTP request — the connection would time out. The upload endpoint immediately returns a `task_id` and delegates processing to FastAPI's `BackgroundTasks`. The frontend polls a `/upload-status/{task_id}` endpoint every 1 second to display a live progress bar. The CSV is read in **10,000-row chunks** using `pandas`, and each chunk is batch-inserted into the database. This keeps memory consumption flat regardless of file size.

Upsert Strategy for WID Uniqueness

The `WID` (Warehouse ID) is the primary key on the `products` table. During CSV processing, each chunk's WIDs are pre-fetched from the database in a single query (`IN` clause). Existing records are updated; new records are bulk-inserted. This ensures WID uniqueness is enforced at the database level and avoids expensive row-by-row `INSERT OR IGNORE` calls.

Two-Stage OCR Pipeline

Product image analysis uses a local-first, cloud-fallback approach:

1. **RapidOCR (ONNX Runtime)** runs entirely locally with no external API calls. It also attempts barcode detection via OpenCV's `BarcodeDetector` .
2. If RapidOCR detects text but cannot extract any structured product fields, **Google Gemini Vision** (`gemini-2.5-flash`) is called as a fallback. Gemini is prompted to return a strict JSON object with product fields.

This design minimizes cloud API costs while providing a robust fallback for complex or low-contrast product labels.

Stateless JWT Authentication + Refresh Token Rotation

- **Access tokens** are short-lived (15 minutes by default) and are never stored in the database.
- **Refresh tokens** are long-lived (7 days), cryptographically random, and stored in the database, enabling server-side revocation on logout.
- The frontend `ApiClient` automatically intercepts 401 responses, attempts a token refresh, and retries the failed request — transparent to the user.

No Build Step Frontend

The frontend uses **Vue 3** (CDN), **Vue Router 4** (CDN), and **Pinia** (CDN) loaded from unpkg. There is no Webpack, Vite, or npm build pipeline. This makes the frontend portable and deployable as plain static files, eliminating Node.js as a deployment dependency entirely.

Technology Stack

Backend

Component	Technology
Web Framework	FastAPI
ASGI Server	Uvicorn
ORM	SQLAlchemy
Data Validation	Pydantic v2
Settings Management	pydantic-settings
Authentication	PyJWT + bcrypt
CSV Processing	Pandas
Local OCR	RapidOCR (ONNX Runtime)
Barcode Detection	OpenCV Contrib
AI Vision (Fallback)	Google Gemini (genai)
PDF Report Generation	fpdf2
Image Processing	Pillow
Timezone Handling	pytz
Async DB Driver	asyncpg
Package Manager	uv

Frontend

Component	Technology
UI Framework	Vue 3 (CDN, no build step)
Routing	Vue Router 4 (CDN)
State Management	Pinia (CDN)

Component	Technology
CSS Framework	Bootstrap 5.3
Icons	Bootstrap Icons 1.11

Database

The system is database-agnostic via SQLAlchemy. It supports:

- **SQLite** (default, zero-config, ideal for development and single-server deployments)
- **PostgreSQL** (recommended for production; set `DATABASE_URL` accordingly)

Project Structure

Product-Verification-System/

```
|
├── backend/                                # Python FastAPI application
│   ├── main.py                            # App entry point, lifespan events, router registration, static files
│   ├── config.py                          # Pydantic Settings model; reads from .env file
│   ├── database.py                        # SQLAlchemy engine, session factory, Base
│   ├── models.py                          # ORM models: User, RefreshToken, Product, Validation
│   ├── schemas.py                         # Pydantic request/response schemas
│   ├── auth_utils.py                      # JWT creation/decoding, bcrypt hashing, RoleChecker dependencies
│   ├── api_utils.py                       # Standardized success_response() and error_response() helpers
│   ├── ocr_utils.py                       # Two-stage OCR pipeline: RapidOCR + Gemini fallback
│   └── routers/
│       ├── auth.py                        # /api/login, /api/signup, /api/refresh, /api/logout
│       ├── products.py                    # /api/products/ – CSV upload, progress polling, WID lookup
│       ├── validations.py                 # /api/validations/analyze – core floor validation endpoint
│       ├── reports.py                     # /api/reports/ – date-range query + CSV/PDF export
│       └── users.py                       # /api/users/ – admin CRUD for user management
├── frontend/                              # Vue 3 SPA (no build step required)
│   ├── index.html                         # Shell HTML; loads CDN scripts, mounts #app
│   └── src/
│       ├── main.js                        # Bootstraps Vue app, Pinia, Vue Router
│       ├── api/
│       │   └── apiClient.js               # Centralized fetch wrapper with auto token-refresh logic
│       ├── router/
│       │   └── index.js                   # Route definitions, navigation guards, role-based redirects
│       ├── store/
│       │   └── authStore.js               # Pinia auth store (tokens, role, username; persisted to localStorage)
│       └── views/
│           ├── Login.js                   # Login form
│           ├── Signup.js                   # Self-registration form (operator/qa roles only)
│           ├── Dashboard.js               # Authenticated shell with sidebar navigation + <router-view>
│           ├── BulkUpload.js              # CSV file picker, background upload, live progress bar, cancel button
│           ├── FloorValidation.js          # WID input, image capture, OCR engine selector, result display
│           ├── QAReports.js               # Date range picker, results table, CSV/PDF export buttons
│           └── UserManagement.js           # User list, create user form, delete user (admin only)
├── test_image/
│   └── image1.png                         # Sample product image for testing OCR pipeline
```

└─ pyproject.toml	# Project metadata and dependencies (uv/pip compatible)
└─ uv.lock	# Locked dependency tree for reproducible installs
└─ .gitignore	# Standard Python gitignore + uploads/ and .env
└─ LICENSE	# Project license

Data Schema

users

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	Internal user ID
username	VARCHAR	UNIQUE, INDEX, NOT NULL	Login identifier
hashed_password	VARCHAR	NOT NULL	bcrypt hash
role	VARCHAR	NOT NULL	admin , operator , or qa
is_active	BOOLEAN	DEFAULT TRUE	Soft-enable/disable flag

refresh_tokens

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	—
token	VARCHAR	UNIQUE, INDEX	Cryptographically random token string
user_id	INTEGER	FK → users.id	Token owner
expires_at	DATETIME	NOT NULL	Token expiry (UTC)

products

Column	Type	Constraints	Description
wid	VARCHAR	PK, INDEX	Warehouse ID — the primary physical item identifier
ean	VARCHAR	INDEX	European Article Number (barcode)

Column	Type	Constraints	Description
mfg_date	VARCHAR	—	Manufacturing date (stored as string to preserve original format)
exp_date	VARCHAR	—	Expiry date (stored as string to preserve original format)

Note: Dates are stored as strings to faithfully represent the data from the CSV. The validation logic normalises separators (/ , . , -) at comparison time, so 10/02/2025 , 10.02.2025 , and 10-02-2025 are treated as equivalent.

validations

Column	Type	Constraints	Description
id	INTEGER	PK, auto-increment	—
wid	VARCHAR	FK → products.wid, INDEX	The WID that was verified
operator_id	VARCHAR	INDEX	Username of the operator who performed the check
timestamp	DATETIME	DEFAULT now() (IST)	When the validation occurred
image_path	VARCHAR	—	Relative path to the uploaded image (e.g. /uploads/uuid.jpg)
gemini_notes	VARCHAR	NULLABLE	Notes from OCR pipeline (success message or error)
status	VARCHAR	NULLABLE	Match , Partial Match , Mismatch , Not Found , Manual Check Required
ocr_method	VARCHAR	NULLABLE	local_ocr or gemini (null if WID-only check)
extracted_data	VARCHAR	NULLABLE	JSON string of fields extracted from the image

Column	Type	Constraints	Description
comments	VARCHAR	NULLABLE	Reserved for future operator comments

API Reference

All endpoints are prefixed with `/api` . The app also serves the frontend at `/` and uploaded images at `/uploads/` . Full interactive docs are available at `/docs` (Swagger UI) when the server is running.

Authentication (`/api`)

Method	Endpoint	Role	Description
POST	<code>/api/signup</code>	Public	Self-register as <code>operator</code> or <code>qa</code> . Admin role cannot be self-assigned.
POST	<code>/api/login</code>	Public	Authenticate with <code>x-www-form-urlencoded</code> credentials; returns access + refresh tokens.
POST	<code>/api/refresh</code>	Public	Exchange a valid refresh token for a new access token.
POST	<code>/api/logout</code>	Authenticated	Invalidate the refresh token server-side.

Products (`/api/products`)

Method	Endpoint	Role	Description
POST	<code>/api/products/upload-csv</code>	admin	Upload a CSV file; processing runs as a background task. Returns a <code>task_id</code> immediately.
GET	<code>/api/products/upload-status/{task_id}</code>	admin	Poll background upload progress (<code>processing</code> / <code>completed</code> / <code>failed</code> / <code>cancelled</code>).

Method	Endpoint	Role	Description
POST	/api/products/cancel-upload/{task_id}	admin	Signal a running upload task to stop after the current chunk.
GET	/api/products/{wid}	admin , operator	Retrieve a single product record by WID.

Validations (/api/validations)

Method	Endpoint	Role	Description
POST	/api/validations/analyze	admin , operator	Submit a WID and/or product image. Runs OCR, looks up the product, compares fields, logs the event, and returns the match status. Request is multipart/form-data with optional wid , image , and ocr_method (local_ocr or gemini) fields.

Validation status values:

Status	Meaning
Match	All comparable fields agree between OCR output and DB record
Partial Match	Some fields match, others don't
Mismatch	No fields match
Not Found	WID provided but not in the database
Manual Check Required	Image provided but OCR could not extract any structured fields

Reports (/api/reports)

Method	Endpoint	Role	Description
GET	/api/reports/	admin , qa	Fetch all validation events in a start_date / end_date range (YYYY-MM-DD).

Method	Endpoint	Role	Description
GET	/api/reports/export	admin , qa	Stream a report download. Add <code>format=csv</code> or <code>format=pdf</code> query param.

Users (/api/users)

Method	Endpoint	Role	Description
GET	/api/users/	admin	List all users (no password hashes returned).
POST	/api/users/	admin	Create a user with any role (admin , operator , qa).
DELETE	/api/users/{user_id}	admin	Delete a user and invalidate all their active sessions. Cannot delete own account.

Role-Based Access Control (RBAC)

RBAC is enforced at **two layers**:

1. Backend — RoleChecker Dependency

`auth_utils.py` defines a `RoleChecker` class that is used as a FastAPI dependency:

```
class RoleChecker:
    def __init__(self, allowed_roles: list):
        self.allowed_roles = allowed_roles

    def __call__(self, payload: dict = Depends(get_current_user_token_payload)):
        role = payload.get("role")
        if role not in self.allowed_roles:
            raise HTTPException(status_code=403, detail=f"Operation not permitted for role: {role}")
        return payload
```

Each router endpoint declares its allowed roles:


```
# Only admins can upload CSV
payload: dict = Depends(auth_utils.RoleChecker(["admin"]))

# Admins and operators can validate products
payload: dict = Depends(auth_utils.RoleChecker(["admin", "operator"]))

# Admins and QA can view/export reports
payload: dict = Depends(auth_utils.RoleChecker(["admin", "qa"]))
```

2. Frontend — Vue Router Navigation Guards

router/index.js checks authStore.role before allowing navigation to any route:

```
router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !isAuthenticated) return next('/login');
  if (to.meta.roles && !to.meta.roles.includes(authStore.role)) return next('/dashboard');
  next();
});
```

Route definitions declare their allowed roles:

```
{ path: 'upload', component: BulkUpload, meta: { roles: ['admin'] } },
{ path: 'validate', component: FloorValidation, meta: { roles: ['admin', 'operator'] } },
{ path: 'reports', component: QAReports, meta: { roles: ['admin', 'qa'] } }
```

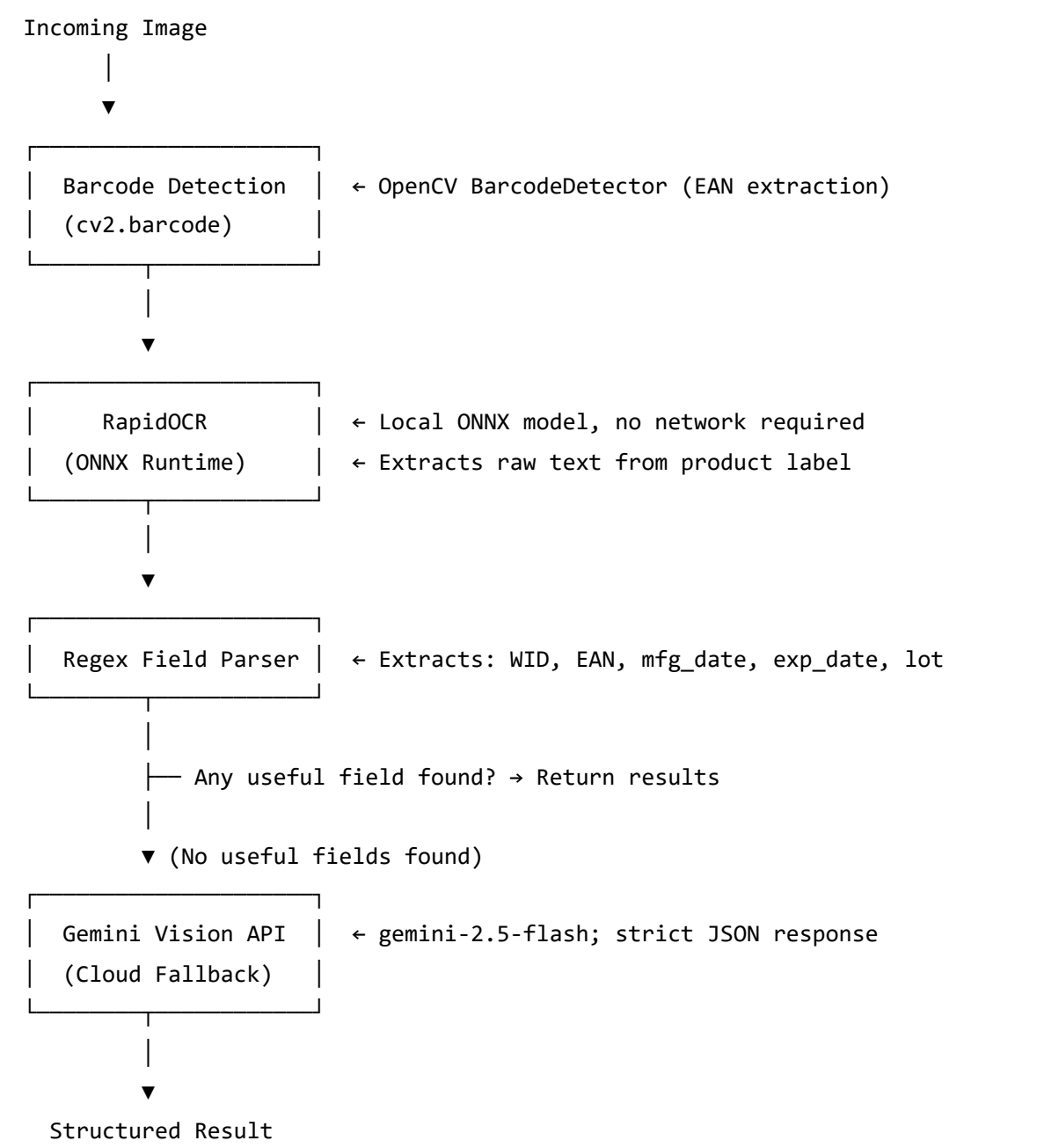
The Dashboard sidebar also conditionally renders navigation items based on authStore.role , so users only see links to pages they are allowed to access.

Access Matrix:

Page / Endpoint	admin	operator	qa
Bulk CSV Upload	✓	✗	✗
Floor Validation	✓	✓	✗
QA Reports	✓	✗	✓
User Management	✓	✗	✗

OCR Pipeline

The OCR system is implemented in `backend/ocr_utils.py` and follows this pipeline:



Regex Patterns Used

Field	Pattern
WID	<code>\bwid\b\s*[:\-\]?\s*([A-Z0-9\-\-]+)</code> (case-insensitive)
EAN	<code>\bean\b\s*[:\-\]?\s*([0-9]{8,14})</code>

Field	Pattern
Mfg Date	(?:mfg mfd manufactured)\s*[:\-\]?\s*(\d{1,2}[/-]\d{1,2}[/-]\d{2,4})
Exp Date	(?:exp expiry expires best before)\s*[:\-\]?\s*(\d{1,2}[/-]\d{1,2}[/-]\d{2,4})
Lot	(?:lot lot no batch)\s*[:\-\]?\s*([A-Z0-9\-\]+)

Date Normalization

Before comparing extracted dates to database dates, the `normalize_date()` function converts all separators to `-` :

- `10/02/2025` → `10-02-2025`
- `10.02.2025` → `10-02-2025`

This ensures consistent matching regardless of how the date appears on the physical label versus how it was entered in the CSV.

Prerequisites

- **Python 3.9+** (3.11+ recommended)
- **uv** package manager (recommended) — or standard `pip`
- A **Gemini API key** (free tier available at [Google AI Studio](#))
- (Optional) **PostgreSQL** for production deployments

Environment Configuration

Create a `.env` file in the project root. This file is listed in `.gitignore` and must **never** be committed.

```
# — Security —————  
# Generate with: python -c "import secrets; print(secrets.token_hex(32))"  
SECRET_KEY=your_super_secret_key_here_change_this  
  
JWT algorithm (leave as HS256 unless you have specific requirements)  
ALGORITHM=HS256  
  
# Token lifetimes  
ACCESS_TOKEN_EXPIRE_MINUTES=15  
REFRESH_TOKEN_EXPIRE_DAYS=7  
  
# — Database —————  
# SQLite (development, zero-config):  
DATABASE_URL=sqlite:///./products.db  
  
# PostgreSQL (production):  
# DATABASE_URL=postgresql://user:password@localhost:5432/product_verification  
  
# — Default Admin Account —————  
# Created automatically on first startup if no users exist.  
DEFAULT_ADMIN_USER=admin  
DEFAULT_ADMIN_PASS=SuperSecureAdmin123!  
  
# — AI / OCR —————  
GEMINI_API_KEY=your_gemini_api_key_here
```

Security Note: Change `DEFAULT_ADMIN_PASS` to a strong password before any deployment. The default credentials are only safe in a local development environment.

Installation & Setup

Option A: Using `uv` (Recommended)

`uv` is a fast Python package manager. The `uv.lock` file in this repo ensures a fully reproducible environment.

```
# 1. Install uv (if not already installed)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Clone the repository
git clone https://github.com/shib1111111/Product-Verification-System.git
cd Product-Verification-System

# 3. Create virtual environment and install all dependencies
uv sync

# 4. Create your .env file
cp .env.example .env # or create it manually as shown above
# Edit .env and fill in your SECRET_KEY and GEMINI_API_KEY
```

Option B: Using pip

```
# 1. Clone the repository
git clone https://github.com/shib1111111/Product-Verification-System.git
cd Product-Verification-System

# 2. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate # Linux / macOS
# .venv\Scripts\activate # Windows

# 3. Install dependencies
pip install -e .

# 4. Create your .env file (see Environment Configuration above)
```

Running the Application

```
# With uv:
uv run uvicorn backend.main:app --reload

# With pip / activated venv:
uvicorn backend.main:app --reload
```

The server starts on <http://127.0.0.1:8000>.

- **Web Application:** <http://127.0.0.1:8000>
- **Interactive API Docs (Swagger UI):** <http://127.0.0.1:8000/docs>
- **Alternative API Docs (ReDoc):** <http://127.0.0.1:8000/redoc>

On first startup, the application will:

1. Create all database tables automatically (`Base.metadata.create_all`).
2. Seed a default admin user from `DEFAULT_ADMIN_USER` / `DEFAULT_ADMIN_PASS` in your `.env` .
or use username as `admin` and password as `SuperSecureAdmin123!` if you haven't changed the `.env` file. Please change these credentials immediately in a production environment to ensure security.
3. Create the `uploads/` directory for storing product images.

Log in with your admin credentials to get started.

CSV Format Specification

The upload CSV must include the following columns (column order does not matter; extra columns are ignored):

Column Name	Required	Description	Example
WID	Yes	Unique Warehouse ID. Primary key. Rows with missing/blank WID are skipped.	WID-001 , A12345
EAN	Yes	European Article Number / barcode. Rows with missing/blank EAN are skipped.	1234567890128
Manufacturing_Date	No	Date of manufacture. Any format.	01-01-2024 , 01/01/2024
Expiry_Date	No	Date of expiry. Any format.	01-01-2026 , 01.01.2026

Example CSV:

WID,EAN,Manufacturing_Date,Expiry_Date
WID-001,1234567890128,01-01-2024,01-01-2026
WID-002,1234567890128,05-03-2024,05-03-2026
WID-003,9876543210987,12-06-2023,12-06-2025

Processing behaviour:

- Rows with blank `WID` or `EAN` are **silently skipped**.
- If a `WID` already exists in the database, its record is **updated** (upsert).
- The file is processed in chunks of 10,000 rows; memory usage stays constant.
- The upload can be **cancelled** mid-process from the UI; rows already committed are retained.

User Guide

For Administrators

First Login:

1. Navigate to the application URL.
2. Log in with the default admin credentials from your `.env`.

Uploading Product Data:

1. Go to **Bulk Upload** in the sidebar.
2. Select your CSV file and click **Upload**.
3. Watch the live progress bar. You can click **Cancel Sync** at any time.

Managing Users:

1. Go to **User Management** in the sidebar.
2. View the list of all users with their roles.
3. Use the **Create User** form to add operators or QA managers.
4. Click the **Delete** button to remove a user (you cannot delete your own account).

Viewing Reports:

Admins have access to the QA Reports page as well. See the QA Manager section below.

For Operators

Validating a Product:

1. Log in with your operator credentials.
2. You will be directed to **Floor Validation** automatically.
3. Either:
 - Type or scan the product's **WID** into the input field, **or**
 - Tap the **image input** to take a photo with your device camera (use `capture="environment"` for the rear camera on mobile), **or**
 - Do **both** for maximum accuracy.
4. Select the **OCR Engine**:
 - **OCR (Local / Fast)** — runs on the server, no extra cost, works for clear labels.
 - **Google Gemini AI (Cloud / Smart)** — better for complex, handwritten, or low-contrast labels.
5. Click **Verify Product**.
6. The result card shows:
 - **Status badge** (Match / Partial Match / Mismatch / Not Found).
 - The product's **database record** (EAN, Mfg Date, Expiry Date).
 - The **extracted data from the image** (if a photo was taken).
 - A preview of the uploaded image.

For QA Managers

Generating a Report:

1. Log in with your QA credentials.
2. You will be directed to **QA Reports** automatically.
3. Select a **Start Date** and **End Date**.
4. Click **Generate Report**.
5. The table displays all validation events in that period, including the operator, timestamp, status, and a thumbnail of the captured image.
6. Click **Export CSV** or **Export PDF** to download the report.

Scalability & Performance

Concern	Solution
Large CSV files	Background task with chunked <code>pd.read_csv(chunksize=10000)</code> keeps memory flat. Live progress polling keeps the user informed.
WID uniqueness under bulk load	Batch pre-fetch of existing WIDs per chunk; bulk insert for new rows; in-place update for existing rows. Single round-trip per chunk.
Database growth (queries)	<code>wid</code> , <code>ean</code> , and <code>timestamp</code> columns are indexed. The reports query uses <code>timestamp</code> range filtering on an indexed column.
Concurrent uploads	<code>UPLOAD_PROGRESS</code> is an in-memory dict keyed by <code>task_id</code> . For multi-worker deployments, this should be migrated to Redis or a database-backed task queue (Celery).
OCR cost management	RapidOCR runs locally with zero API cost. Gemini is only called when local OCR yields no structured fields.
Token expiry handling	Short-lived access tokens (15 min) limit exposure. The frontend <code>ApiClient</code> handles silent refresh, so the user is never interrupted mid-session.

Recommended production upgrades for very high scale (>10M records):

- Switch `DATABASE_URL` to PostgreSQL.
- Use a PGVECTOR or native PostgreSQL index for `wid` lookups.
- Replace `BackgroundTasks` with Celery + Redis for distributed CSV processing.
- Add a CDN (e.g., Cloudflare R2 or AWS S3) for uploaded product images.

Security Considerations

- **Passwords** are hashed with `bcrypt` (auto-salted). Plain-text passwords are never stored.
- **JWT secrets** must be cryptographically random and kept in `.env` (never committed to source control).
- **Refresh tokens** are stored in the database, enabling server-side revocation on logout. Expired tokens are cleaned up on access.

- **Role enforcement** happens on the backend for every protected endpoint — the frontend RBAC is a UX convenience, not a security boundary.
- **Self-signup** is restricted to operator and qa roles; admin role can only be assigned by an existing admin through the User Management page.
- **Admin self-deletion** is blocked to prevent accidental logout.
- **CORS** is currently set to `allow_origins=["*"]` which is appropriate for development. In production, restrict to your specific frontend domain.
- **File uploads** are stored in a local `uploads/` directory. For production, validate MIME types and consider moving to object storage.

Troubleshooting

422 Unprocessable Entity on login

The login endpoint uses `application/x-www-form-urlencoded` (OAuth2 standard), not JSON. Ensure your client sends the correct `Content-Type` header.

OCR always returns Manual Check Required

- Verify the image is clear and well-lit.
- Check that `GEMINI_API_KEY` is set correctly in `.env`.
- Try switching the OCR Engine to **Google Gemini AI** in the validation form.
- Confirm the label contains recognizable keywords like `MFG: , EXP: , WID: .`

CSV upload fails immediately with CSV is missing required columns

- Ensure column headers are exactly `WID , EAN , Manufacturing_Date , Expiry_Date` (case-sensitive).
- Leading/trailing whitespace in headers is automatically stripped by the backend.

MIME type mismatch errors in browser for JS files

This is a known Windows issue where the OS registry may map `.js` to `text/plain`. The `main.py` startup code explicitly fixes this:

```
mimetypes.add_type("application/javascript", ".js")
```

If you still see the issue, check your Windows registry under `HKEY_CLASSES_ROOT\.js` .

Database `OperationalError: no such table`

The tables are created automatically on startup via `Base.metadata.create_all(bind=engine)` . If tables are missing, ensure the application started without errors. Check the console output for any SQLAlchemy errors.

RefreshToken `timezone comparison warning`

The refresh token expiry check compares timezone-aware datetimes. The code explicitly adds UTC timezone info:

```
if db_token.expires_at.replace(tzinfo=timezone.utc) < datetime.now(timezone.utc):
```

This handles SQLite's naive datetime storage correctly.

Uploaded images not displaying in reports

Ensure the `uploads/` directory exists and is writable. The app creates it on startup, but verify file system permissions in containerized environments.

License

This project is licensed under the terms of the [LICENSE](#) file included in this repository.

Contributing

This project was built as an end-to-end assignment deliverable. If you are reviewing or extending this codebase, key areas for enhancement include:

- **Celery + Redis** for distributed background task processing.
- **Alembic** for database migration management.

- **Pytest** suite for endpoint and OCR unit testing.
- **Object storage** (S3/R2) for production image storage.
- **WebSocket support** for real-time upload progress (replacing polling).
- **Audit logging** for admin user management actions.

Built with FastAPI, Vue 3, SQLAlchemy, RapidOCR, and Google Gemini.

Contact

Developed by Shib Kumar Saraf

For enterprise inquiries or contributions, please reach out via GitHub.

 **Email:** shibkumarsaraf05@gmail.com

 **GitHub:** github.com/shib1111111